# Concurrency patterns

Task 6. A party is a small shared state that many phones change at the same time, so races are normal here. This document picks six places where concurrency or distributed patterns are used. For each one: the situation, the pattern, a sequence diagram, and where it lives in the code.

| # | Situation | Pattern |
|---|---|---|
| 1 | Two guests add the same song at the same moment | Per-room lock (aggregate serialization) |
| 2 | The response to "add" is lost and the client retries | Idempotency key (store and replay) |
| 3 | Several skip votes arrive together | Decide under lock + guarded transition (compare-and-set) |
| 4 | Two tabs refresh the same token, or a thief replays it | Optimistic concurrency + reuse detection |
| 5 | Many tracks need slow LLM judging | Bounded queue + micro-batching |
| 6 | Ten votes in 50 ms, events arrive out of order | Coalescing pub/sub + monotonic versions |

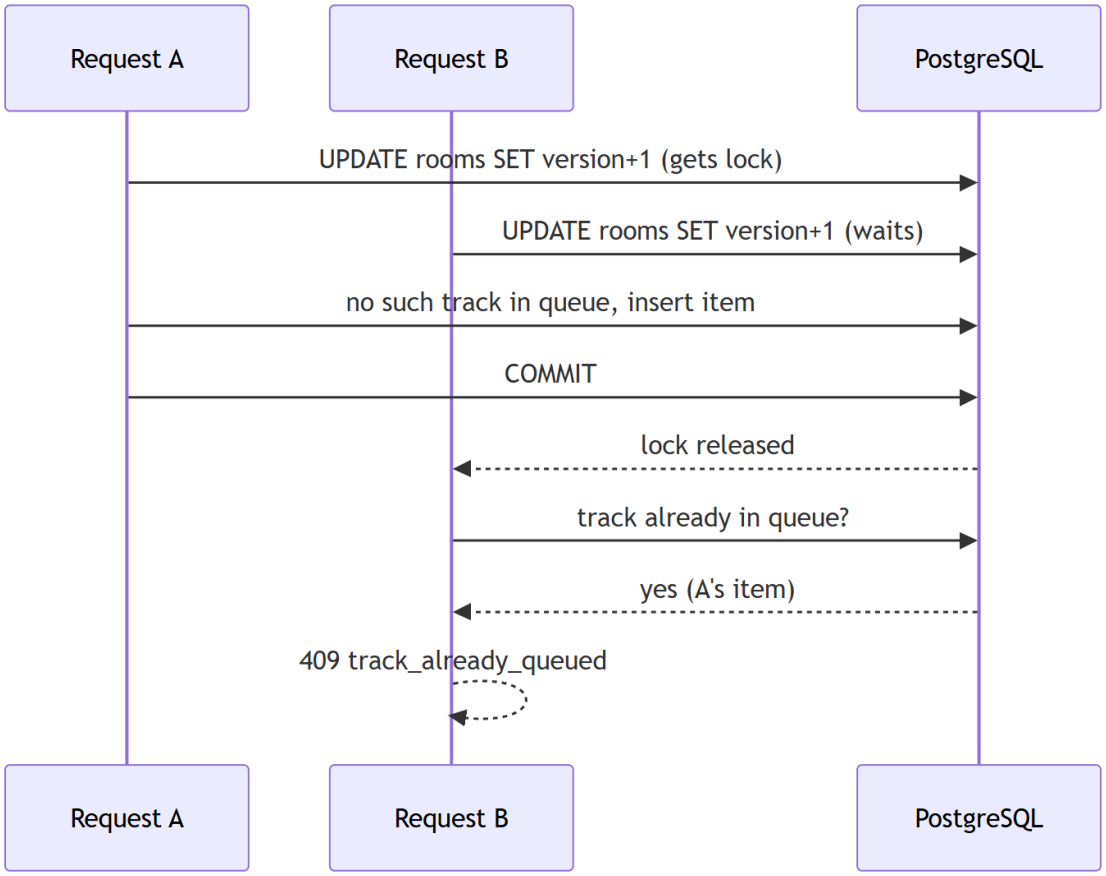
## 1. Per-room lock

**Situation.** Guest A and guest B tap "+" on the same track within a few milliseconds. Both checks "is it already queued?" would pass and the track would be queued twice. Parallel adds could also get around the per-guest limit.

**Pattern.** The room is the unit of consistency. Every transaction that changes a room starts with

```
UPDATE rooms SET version = version + 1 WHERE id = @roomId RETURNING version;
```

The row lock from this statement is held until commit, so the second request waits, then reads the fresh data. The version also serves as the order number for realtime events (flow 6). Slow work (Deezer, LLM) is done before taking the lock, so the lock is held only a few milliseconds.

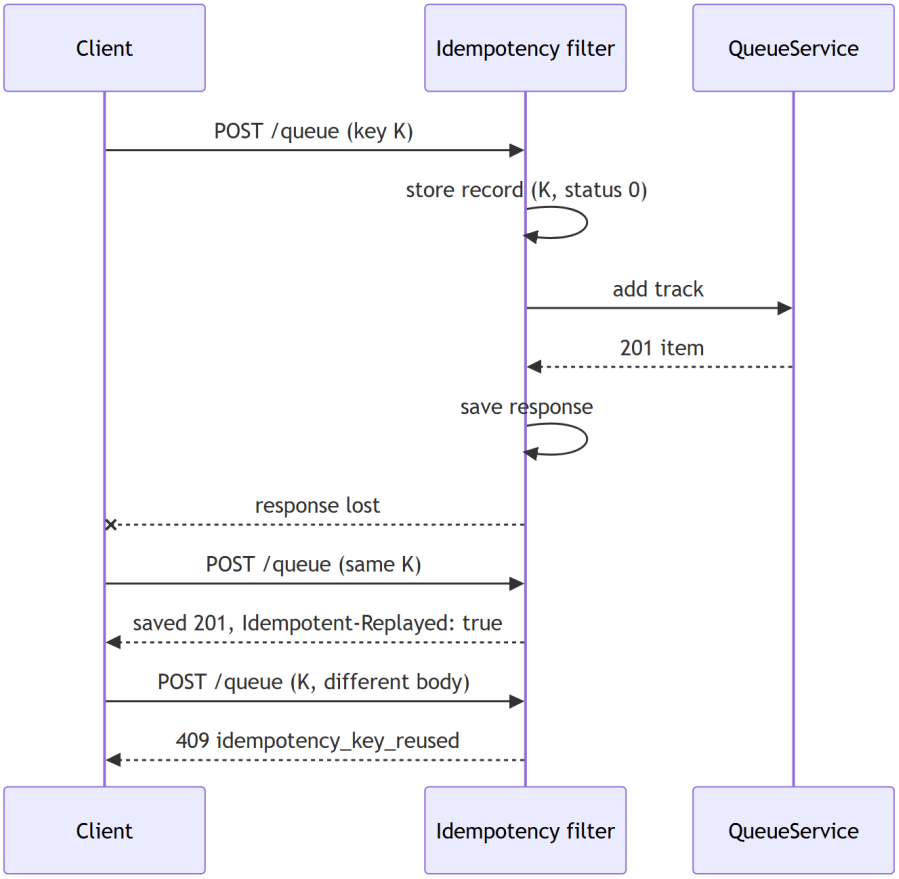



Code: `LockRoomAsync` in the DB context, called by every mutating use case. A partial unique index (`ux_queue_unique_waiting_track`) is the safety net if some path forgets the lock.

## 2. Idempotency key

**Situation.** The phone is on bad Wi-Fi. The add request reaches the server, but the response is lost. The user taps again.

**Pattern.** The client sends an `Idempotency-Key`. The server stores (user, key, request hash) before doing the work and saves the response afterwards. A retry returns the saved response. The primary key (user, key) makes the first insert act as a mutex.

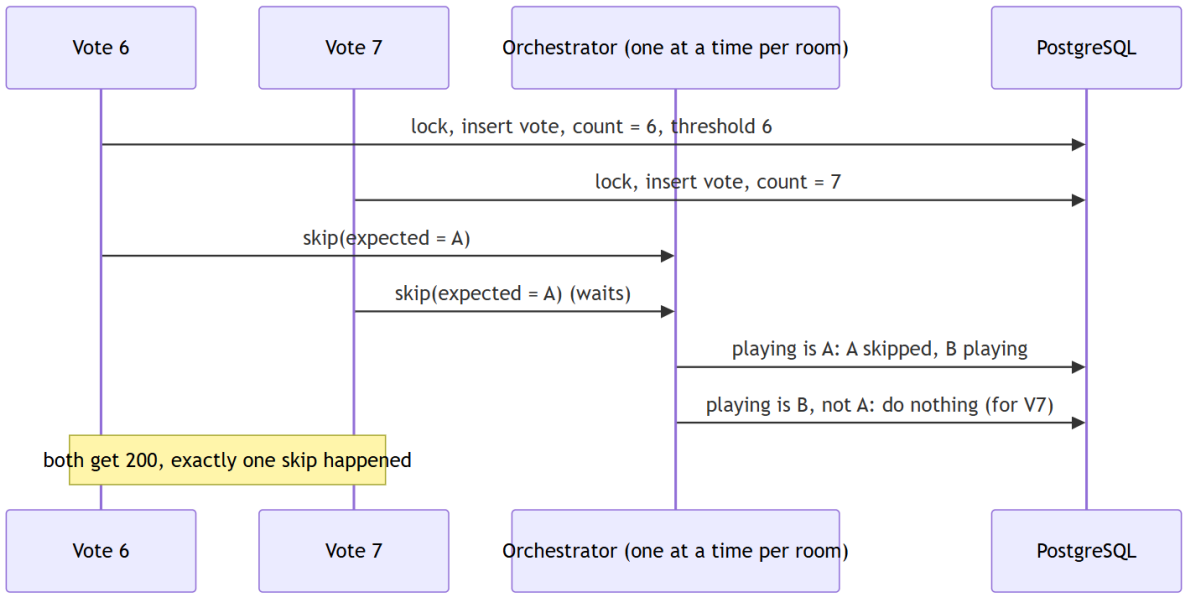



If the same key arrives while the first request is still running, the answer is 409 `request_in_progress`. Failed requests delete their record so the client can retry. Records live 24 hours.

## 3. Exactly-once Fair Skip

**Situation.** Ten listeners, so six votes are needed. Votes 6, 7 and 8 arrive within 50 ms. Each request sees "threshold reached". Without care the song would be skipped three times and two songs would be lost.

**Pattern.** Two steps. First, record and count the vote under the room lock. Second, skip with the item the caller saw (`SkipAsync(expectedItemId)`). If that item is not the playing one any more, nothing happens. This is a compare-and-set on the "now playing" slot, run one at a time per room.

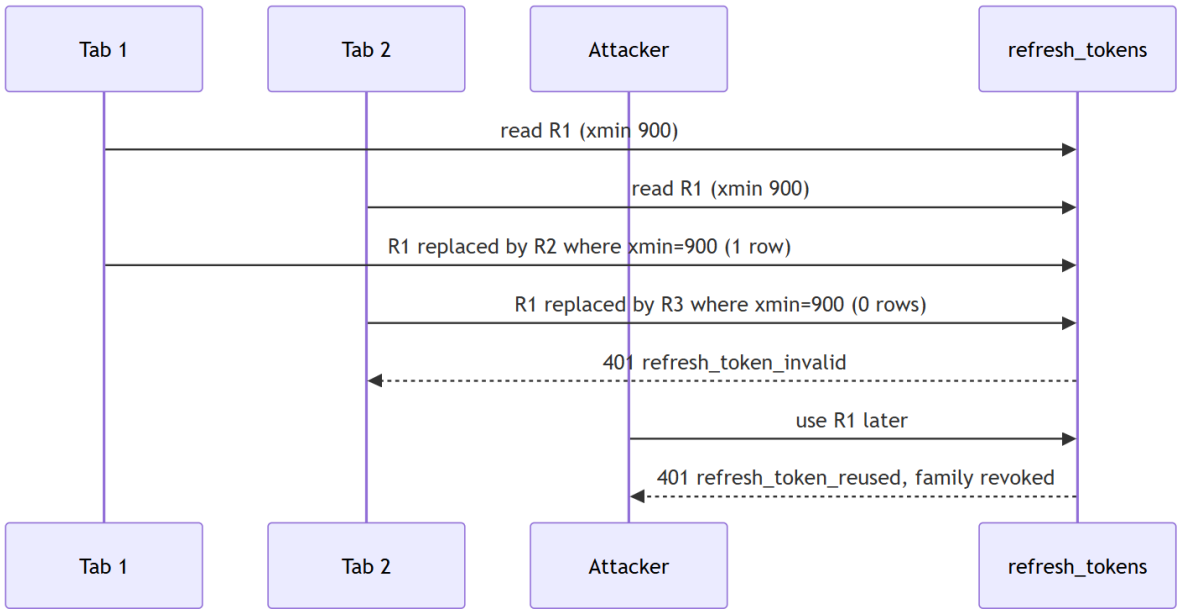



Code: `PlaybackService.CastSkipVoteAsync`, `PlaybackOrchestrator`. Unique indexes `ux_queue_one_playing` and `ux_queue_one_next` make a second "playing" item impossible even if the code was wrong.

## 4. Refresh token rotation

**Situation.** Two browser tabs refresh with the same token at once. Or an attacker replays a token that the real device already used.

**Pattern.** Optimistic concurrency on the PostgreSQL row version (`xmin`): both tabs read the token, both try to mark it as replaced, only one update affects a row. For the attacker, a token that already has a successor must never be seen again, so the whole token family is revoked. On the client, refresh is single-flight (one request shared by all callers).

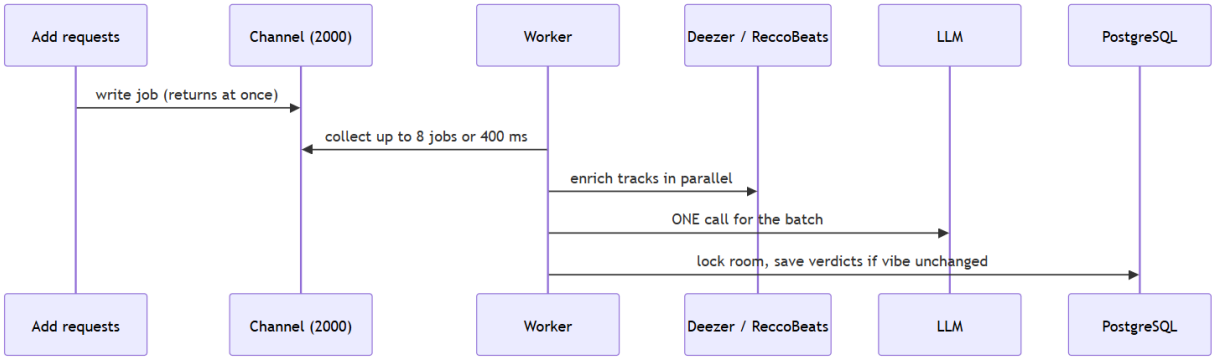


Known gap: the losing tab currently treats its 401 as a lost session and registers a new anonymous user. The planned fix is to re-read the token from storage first (or use the Web Locks API).

## 5. Bounded queue with micro-batching

**Situation.** 30 guests add songs in 10 seconds. Each song needs details from Deezer, audio features, and an LLM verdict. The free LLM takes 2-4 seconds per call and has a daily limit. User requests must never wait for it.

**Pattern.** Producers write jobs into a bounded in-memory channel without blocking (oldest dropped when full). One worker takes up to 8 jobs or waits at most 400 ms, enriches the tracks in parallel, and sends one LLM call for the whole batch. Before saving, the result is checked against the current vibe, and dropped if the host changed the vibe meanwhile.

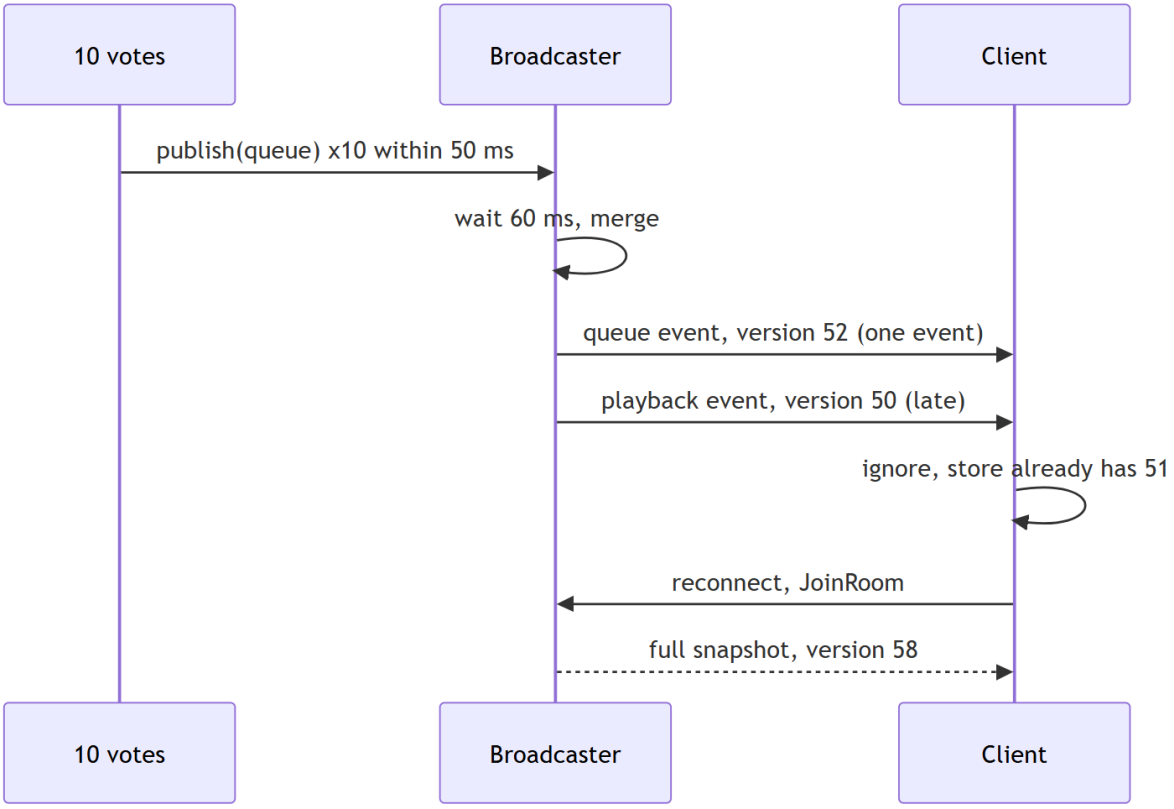



Five tracks cost one LLM call instead of five. The backlog is exported as a metric (`quefy_pipeline_backlog`).

## 6. Coalescing events with versions

**Situation.** A song gets ten votes in 50 ms. Sending ten events to 150 phones is wasteful. Events can also arrive out of order, or be missed when a phone loses its connection.

**Pattern.** The broadcaster collects "what changed" for 60 ms and sends one event with the whole changed part. Every event carries the room version. A client applies an event only if its version is not older than what it has. After every reconnect the client joins again and receives a full snapshot.




## Other places

Room lock + guarded transitions also cover the AI DJ autofill (in-flight flag, 120 s cooldown, atomic budget counter), single-use pairing codes (checked again under the lock), settings edits (optimistic `expectedRevision`), the catalog cache (single request per query) and presence (leases refreshed by heartbeat).

## Scaling out

On one node, the room lock and in-memory gates are enough. With several nodes the room lock still works (it is in the database). Presence, add cooldown buckets and SignalR fan-out must move to Redis, and a speaker's room must be handled by the node that holds its connection.